

DELETE OPERATIONS

Delete First Node (Concept)

Steps:

- Move head to next node

Visual:

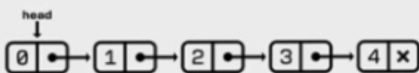
Before:

Head → [10] → [20] → [30]

After:

Head → [20] → [30]

Data Structures Linked List Deletion



< T 0 >

DELETE FIRST NODE (PSEUDOCODE)



IF head != null:
head = head.next

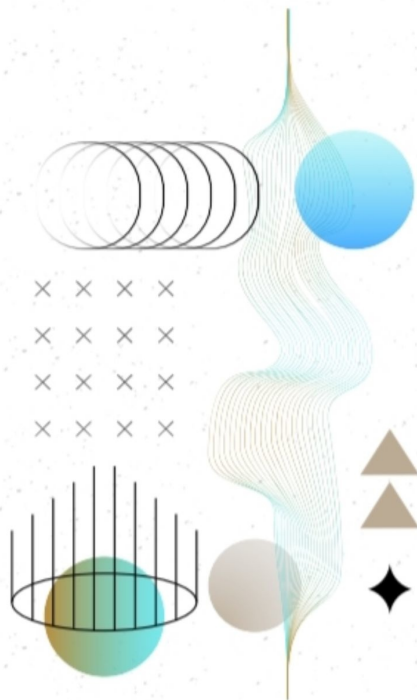
DELETE FIRST NODE (C#)



```
Node DeleteFirst(Node head)
{
    if (head == null)
        return null;

    head = head.next;
    return head;
}
```

DELETE AT END (CONCEPT)



Steps:

- Traverse to second last node
- Set its next to null

Visual:

Before:

$[10] \rightarrow [20] \rightarrow [30]$

After:

$[10] \rightarrow [20]$



DELETE AT END (PSEUDOCODE)



IF head == null:
return

current = head

WHILE current.next.next != null:
current = current.next

current.next = null



DELETE FIRST NODE (C#)

```
Node DeleteFirst(Node head)
{
    if (head == null)
        return null;

    head = head.next;
    return head;
}
```



DELETE AT END (CONCEPT)

Steps:

- Traverse to second last node
- Set its next to null

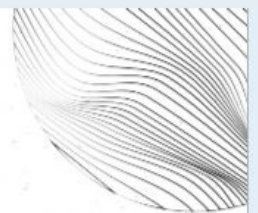
Visual:

Before:

[10] → [20] → [30]

After:

[10] → [20]



DELETE AT END (PSEUDOCODE)



IF head == null:
return

current = head

WHILE current.next.next != null:
current = current.next

current.next = null



DELETE AT END (C#)

NODE DELETEATEND(NODE HEAD)

```
{  
  IF (HEAD == NULL || HEAD.NEXT == NULL)  
    RETURN NULL;  
}
```

NODE CURRENT = HEAD;

```
WHILE (CURRENT.NEXT.NEXT != NULL)
```

```
{  
  CURRENT = CURRENT.NEXT;  
}
```

```
{  
  CURRENT = CURRENT.NEXT;
```

```
  CURRENT.NEXT = NULL;
```

```
  RETURN HEAD;  
}
```



DELETE AT END (PSEUDOCODE)



IF head == null:
return

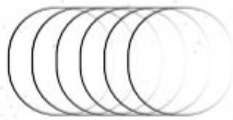
current = head

WHILE current.next.next != null:
current = current.next

current.next = null



DELETE AT END (C#)



NODE DELETEATEND(NODE HEAD)

```
{  
  IF (HEAD == NULL || HEAD.NEXT == NULL)  
    RETURN NULL;  
  CURRENT = HEAD;  
  WHILE (CURRENT.NEXT.NEXT != NULL)  
    CURRENT = CURRENT.NEXT;  
  CURRENT.NEXT = NULL;  
  RETURN HEAD;  
}
```

NODE CURRENT = HEAD;

WHILE (CURRENT.NEXT.NEXT != NULL)

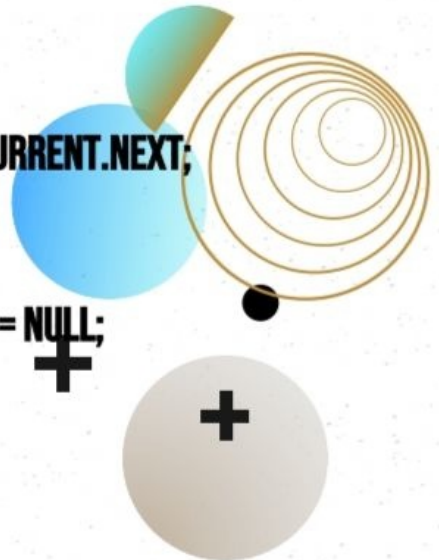
x x x x
x x x x
x x x x
x x x x



CURRENT = CURRENT.NEXT;

CURRENT.NEXT = NULL;

RETURN HEAD;





DELETE IN THE MIDDLE (CONCEPT)

Steps:

- Traverse to previous node
- Skip the target node

Visual:

Before:

[10] → [20] → [30] → [40]

Delete 30

After:

[10] → [20] → [40]



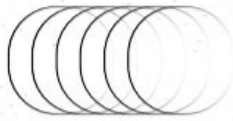
DELETE IN THE MIDDLE (PSEUDOCODE)

current = head

WHILE current.next.data != target:
 current = current.next

current.next = current.next.next





DELETE IN THE MIDDLE

(C#)

```
NODE DELETEVALUE(NODE HEAD, INT VALUE)
{
```

```
    IF (HEAD == NULL)
        RETURN NULL;
```

```
    IF (HEAD.DATA == VALUE)
        RETURN HEAD.NEXT;
```

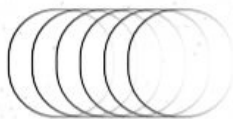
```
    NODE CURRENT = HEAD;
    x x x x
    x x x x
```

```
    WHILE (CURRENT.NEXT != NULL &&
CURRENT.NEXT.DATA != VALUE)
```

```
    {
        CURRENT = CURRENT.NEXT;
    }
```

```
    IF (CURRENT.NEXT != NULL)
    {
```

```
        CURRENT.NEXT = CURRENT.NEXT.NEXT;
    }
```



DELETE IN THE MIDDLE

(C#)

```
NODE DELETEVALUE(NODE HEAD, INT VALUE)
{
```

```
    IF (HEAD == NULL)
        RETURN NULL;
```

```
    IF (HEAD.DATA == VALUE)
        RETURN HEAD.NEXT;
```

```
    NODE CURRENT = HEAD;
    x x x x
    x x x x
```

```
    CURRENT = CURRENT.NEXT;
```

```
    IF (CURRENT.NEXT != NULL)
```

```
    {
        CURRENT.NEXT = CURRENT.NEXT.NEXT;
    }
```

```
    RETURN HEAD;
}
```

